5.4 설명 가능한 CNN



EWHA EURON

#01 설명 가능한 CNN



정의

- 딥러닝 처리 결과를 **사람이 이해할 수 있는 방식**으로 제시하는 기술

필요성

- CNN의 예측 결과만으로는 내부 처리 과정을 이해하고 신뢰하기 어려움

-> **중간계층을 시각화**하면 입력 특징의 변화 확인 가능

시각화 방법

- **필터**에 대한 시각화 : 모델의 가중치 패턴 확인
- **특성 맵**에 대한 시각화 : 특정 입력에 대한 중간 계층의 반응 확인

EWHA EURON

#02 특성 맵 시각화

특성 맵(feature map)이란?

- 입력 이미지 또는 다른 특성 맵에 **필터를 적용한 결과**
- 활성화 맵이라고도 부름
- 각 필터는 이미지의 **서로 다른 특징에 반응**

-> 특성 맵을 통해 CNN이 이미지의 **어떤 특징을 학습하는지 확인 가능**

시각화의 목적

- 입력 이미지가 합성곱 계층을 거치면서 어떻게 변화하는지 확인
- 특성 맵에서 입력 특성을 감지하는 방법을 이해하는 데 도움

입력 이미지 -> Conv -> Feature Map

EWHA EURON

#03 라이브러리 호출

라이브러리 설치

> pip install pillow

```
import matplotlib.pyplot as plt
from PIL import Image
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

EWHA EURON

#04 네트워크 생성

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), ------ inplace=True는 기존의 데이터를 연산의
            nn.Dropout(0.3),                결괏값으로 대체하는 것을 의미
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(512, 512, bias=False),
            nn.Dropout(0.5),
            nn.BatchNorm1d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(-1, 512)
        x = self.classifier(x)
        return F.log_softmax(x) ------ ①
```

#05 네트워크 생성

네트워크 구성

- 합성곱층 13개, 완전연결층 2개
- 활성화 함수: ReLU

```
self.features = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=3, bias=False),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    nn.Dropout(0.3),
    nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),
```

inplace=True는 기존의 데이터를 연산의 결괏값으로 대체하는 것을 의미

1) 특징 추출

- 여러 합성곱층을 순차적으로 구성
- Conv2d -> BatchNorm2d -> ReLU 구조 반복
- 일부 구간에 Dropout, MaxPool2d 적용

```
self.classifier = nn.Sequential(
    nn.Linear(512, 512, bias=False),
    nn.Dropout(0.5),
    nn.BatchNorm1d(512),
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(512, num_classes)
)
```

2) 분류

- 추출된 특징 입력받아 **최종 클래스 분류**
- Linear -> Dropout -> BatchNorm1d -> ReLU
- 마지막 Linear 층에서 출력 차원을 num_classes로 출력

EWHA EURON

#06 네트워크 생성

```
def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x) ------ ①
```

3) 순전파 과정

- 합성곱층 통과하며 **특징 추출**
- 완전연결층 통해 **분류**
- 최종 출력값 반환

로그 소프트맥스

- 소프트맥스 결과에 log 값을 취한 연산
- 신경망 마지막 출력을 **로그 확률**로 표현
-> 기본 소프트맥스의 **기울기 소멸 문제** 보완

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\text{logsoftmax} = \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right)$$

$$= x_i - \log\left(\sum_{j=1} e^{x_j}\right)$$

EWHA EURON

#07 모델 객체화

```
model = XAI() ------ model이라는 이름의 객체를 생성
model.to(device) ------ model을 장치(CPU 혹은 GPU)에 할당
model.eval() ------ 테스트 데이터에 대한 모델 평가 용도로 사용
```

model.eval()

- 학습모드 -> **평가모드**
- **Dropout 비활성화** : 모든 뉴런 사용
- BatchNorm : 현재 배치 대신 학습 중 저장한 값으로 정규화

-> 평가 시 **일관된 출력값**을 얻기 위함

EWHA EURON

#08 특성 맵 확인 클래스 정의

- 특정 합성곱층의 출력값 확인
- 순전파 과정의 중간 결과를 가져와 시각화

```
class LayerActivations:
    features = []
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn) ------ ①

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self): ------ hook 삭제
        self.hook.remove()
```

코드 설명

- register_forward_hook(): 원하는 계층의 출력값 가져오기
- detach(): 출력값을 계산 그래프에서 분리하여 저장
- self.features: 가져온 특성 맵 저장
- remove(): 사용 끝난 hook 제거

EWHA EURON

#09 특성 맵 확인 클래스 정의

중간 변수와 Hook

```
import torch
x = torch.Tensor([0,1,2,3]).requires_grad_()
y = torch.Tensor([4,5,6,7]).requires_grad_()
w = torch.Tensor([1,2,3,4]).requires_grad_()
z = x + y;
o = w.matmul(z)
o.backward()
print(x.grad, y.grad, z.grad, w.grad, o.grad)
```

```
tensor([2., 3., 4., 5.]) tensor([2., 3., 4., 5.]) None tensor([ 4.,  6.,  8., 10.]) None
```

- 직접 생성한 텐서(x, y, w)는 기울기 확인 가능
- 중간 변수(z, o)는 기울기 저장 X

-> z.register_hook(hook_fn) 사용 시 중간 계층의 기울기 확인 가능

EWHA EURON

#10 이미지 호출

```
img = cv2.imread("../chap05/data/cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR) ------①
img = ToTensor()(img).unsqueeze(0) ------②
print(img.shape)
```

cv2.resize : 이미지 변경

- 이미지 크기를 100 x 100으로 변경
- interpolation: 보간법

보간법이란?

: 이미지 크기 변경할 때, 새롭게 필요한 픽셀 값을 추정해서 채우는 방법

- 기존 픽셀 데이터에 근사 함수 $f(x, y)$ 적용
- 확대 -> 새로 생기는 위치에 픽셀 값 매핑
- 축소 -> 여러 픽셀 정보를 압축하여 새로운 값으로 할당

EWHA EURON

#11 이미지 호출

```
img = cv2.imread("../chap05/data/cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR) ------ ①
img = ToTensor()(img).unsqueeze(0) ------ ②
print(img.shape)
```

ToTensor().unsqueeze(0) : Tensor 변환

- 이미지를 Tensor로 변환
- 맨 앞에 배치 차원 추가

unsqueeze()란?

: 텐서에 크기가 1인 새로운 차원을 추가하는 함수

```
import torch

x1 = torch.rand(3, 10, 64)

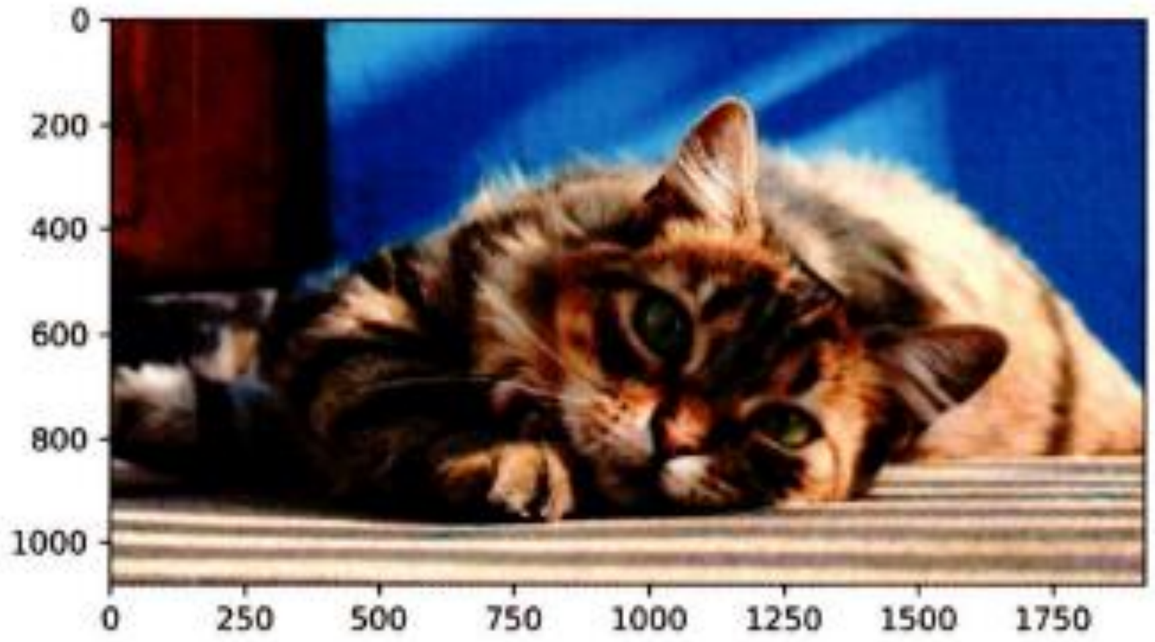
x2 = x1.unsqueeze(dim=0) ------ [3, 10, 64] -> [1, 3, 10, 64]
print(x2.shape)
print('---------------')
x3 = x1.unsqueeze(dim=1) ------ [3, 10, 64] -> [3, 1, 10, 64]
print(x3.shape)
```

```
torch.Size([1, 3, 10, 64])
---------------
torch.Size([3, 1, 10, 64])
```

EWHA EURON

#12 이미지 호출

예제에서 사용할 이미지



#13 첫 번째 계층 특성 맵

```
result = LayerActivations(model.features, 0) ------ 0번째 Conv2d 특성 맵 확인

model(img)
activations = result.features

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()

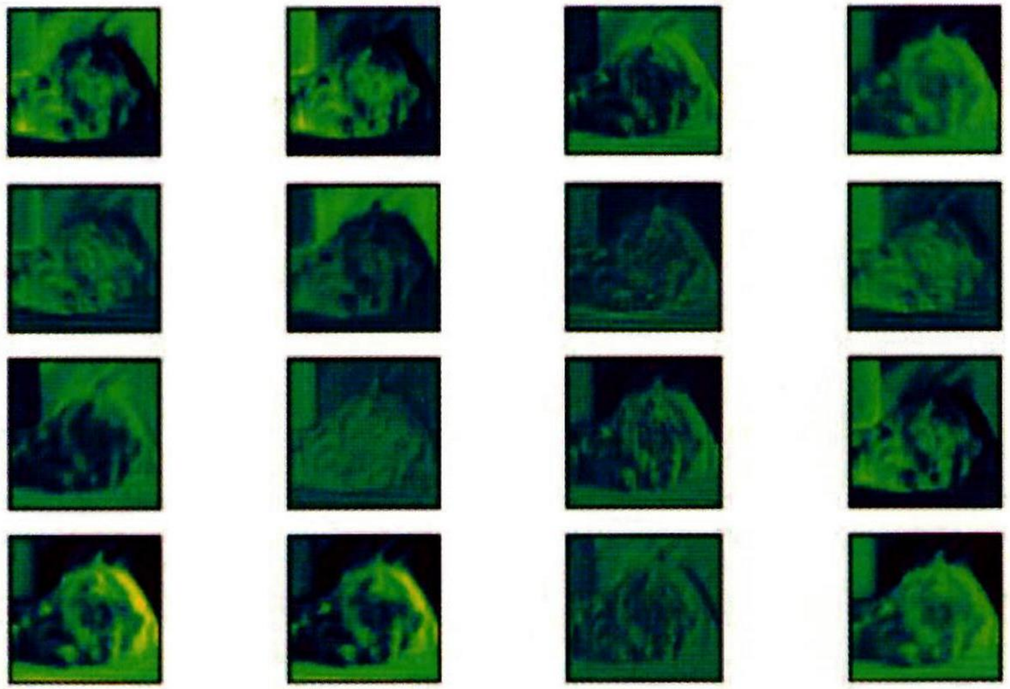

fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
```

EWHA EURON

#14 첫 번째 계층 특성 맵

```
result = LayerActivations(model.features, 0) ------ 0번째 Conv2d 특성 맵 확인

model(img)
activations = result.features
```



결과

- 입력층과 가까운 계층
- 입력 이미지의 형태가 많이 유지됨

EWHA EURON

#15 20번째 계층 특성 맵

```
result = LayerActivations(model.features, 20) ····· 20번째 Conv2d 특성 맵 확인

model(img)
activations = result.features

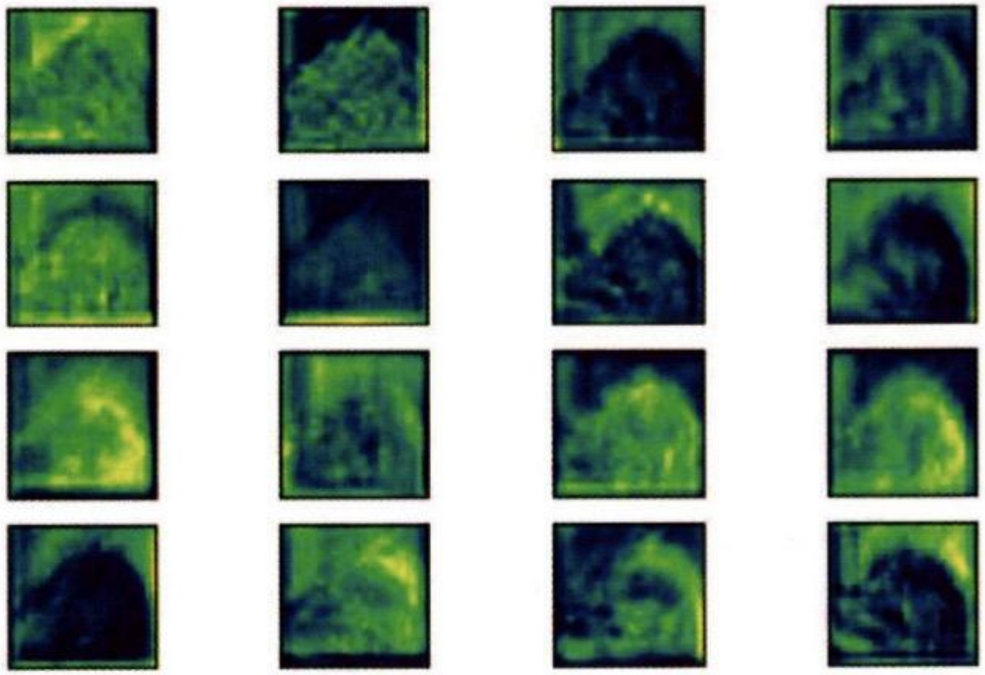
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

EWHA EURON

#16 20번째 계층 특성 맵

```
result = LayerActivations(model.features, 20) ····· 20번째 Conv2d 특성 맵 확인

model(img)
activations = result.features
```



결과

- 이미지 형태가 점차 변화
- 특징이 더 강조됨

EWHA EURON

#17 40번째 계층 특성 맵

```
result = LayerActivations(model.features, 40) ------ 40번째 Conv2d 특성 맵 확인

model(img)
activations = result.features

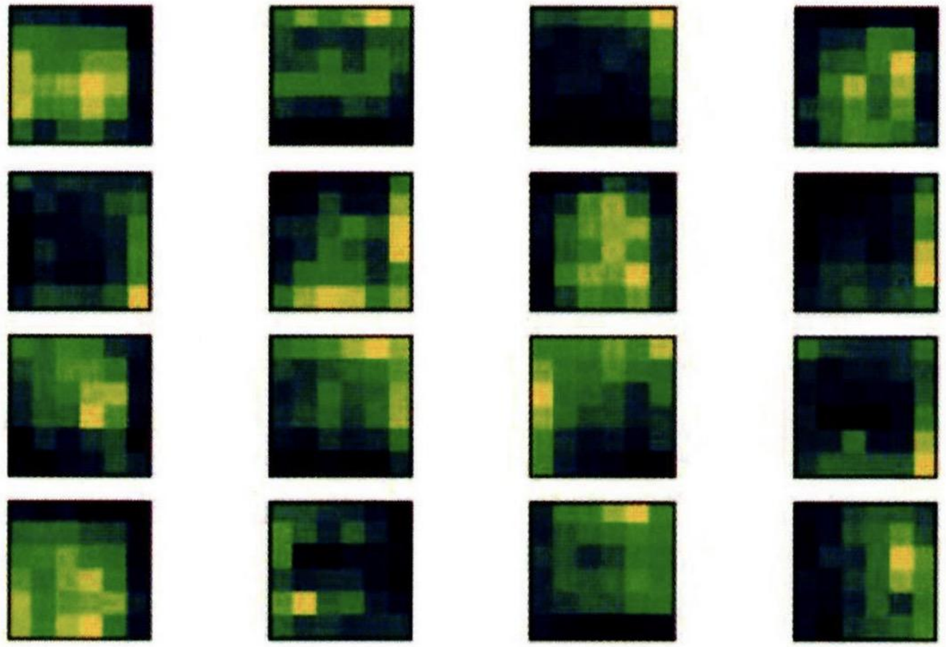
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
```

EWHA EURON

#18 40번째 계층 특성 맵

```
result = LayerActivations(model.features, 40) ------ 40번째 Conv2d 특성 맵 확인

model(img)
activations = result.features
```



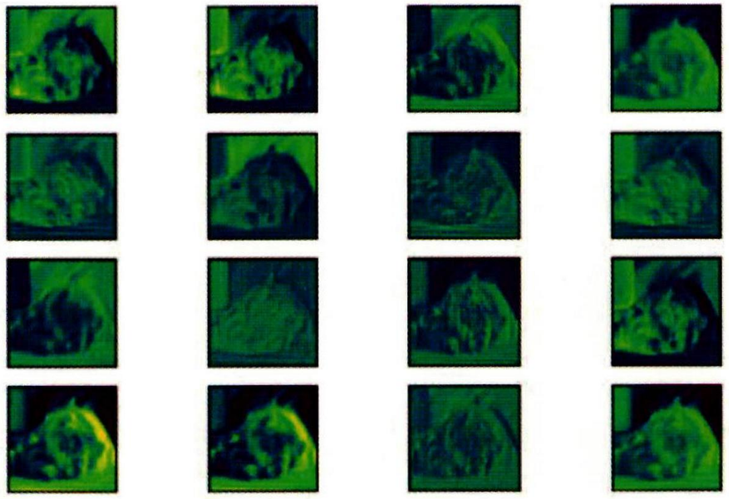
결과

- 원래 형태를 거의 확인하기 어려움
- 이미지 특징 중심으로 전달

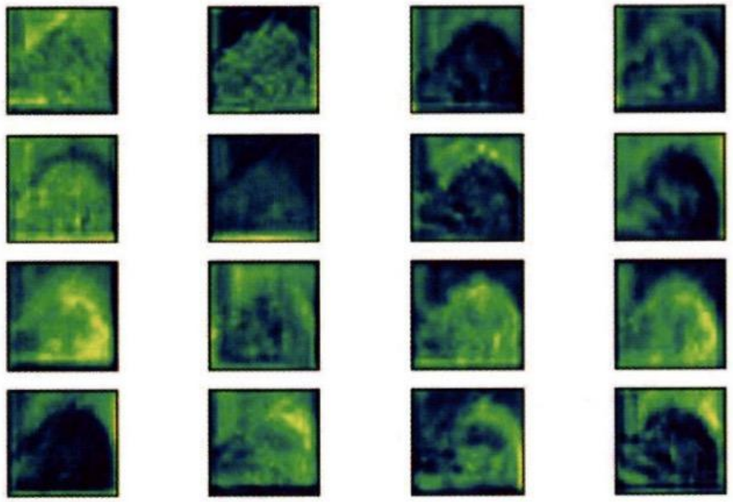
EWHA EURON

#19 계층별 비교

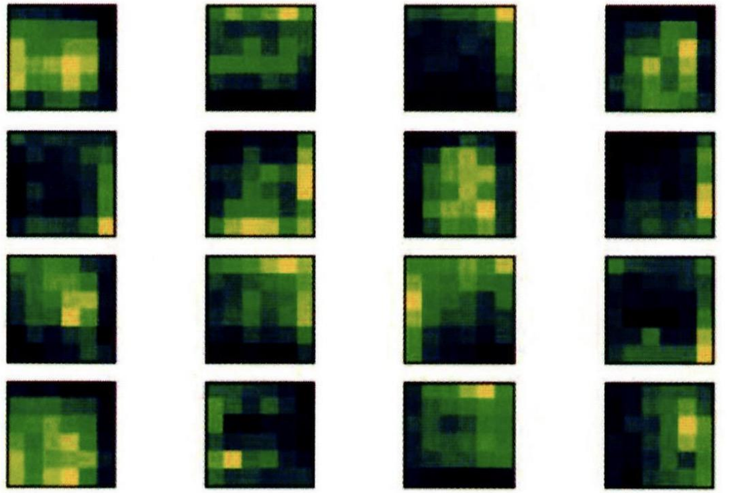
첫 번째 계층(0)



20번째 계층



40번째 계층



출력 결과 해석

- 출력층에 가까워질수록 **원래 이미지 형태는 사라짐**
- **학습된 특징이 중심**이 되어 전달됨

EWHA EURON

#20 설명 가능한 CNN

신뢰성 문제

- 최근 딥러닝 결과에 대한 **신뢰성 문제** 제기되며, 설명 가능한 인공지능(XAI) 이슈가 됨
- CNN은 필터와 특성 맵을 시각화하여 내부 처리 과정을 확인

-> 모델의 판단 근거를 이해하고 결과의 신뢰성 확보에 활용

EWHA EURON

5.5 그래프 합성곱 네트워크



EWHA EURON

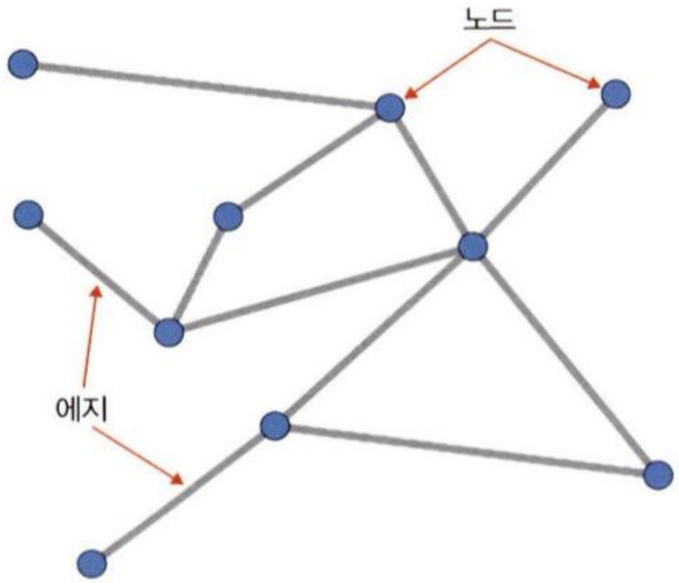
#01 그래프란?

그래프

: 방향성이 있거나(directed) 없는(undirected) 에지로

연결된 노드의 집합

- 노드, 에지는 문제에 대한 지식, 직관 바탕으로 구성



구성 요소

- **노드**(node, vertex)

: 그래프에서 대상을 나타내는 점 = 원소

- **에지**(edge)

: 두 노드를 연결한 선 = 결합 방법

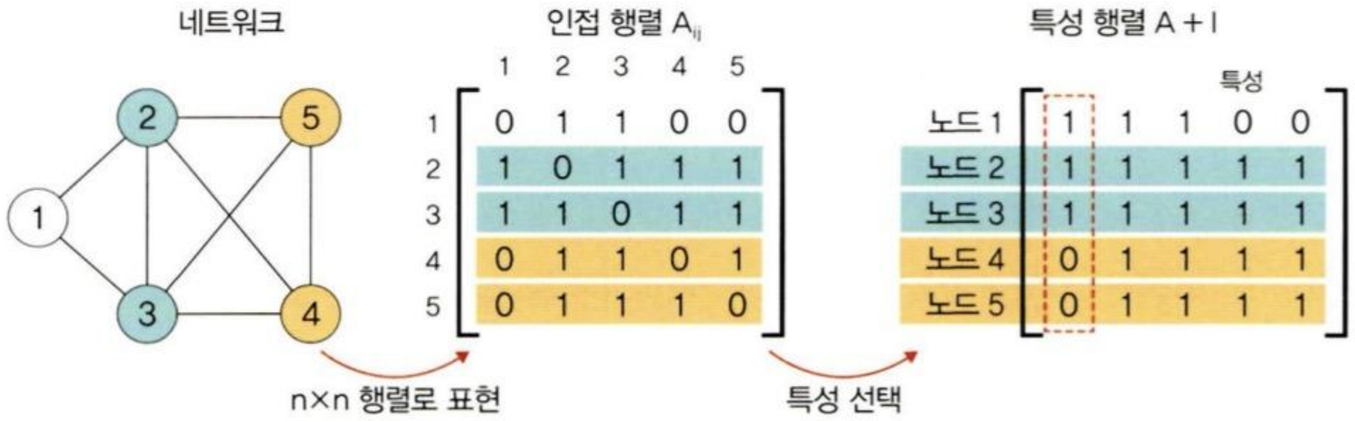
EWHA EURON

#02 그래프 신경망

그래프 신경망(Graph Neural Network, GNN)이란?

: 그래프 구조에서 사용하는 신경망

- 그래프 데이터를 컴퓨터가 이해할 수 있도록 행렬 형태로 표현



#03 그래프 신경망

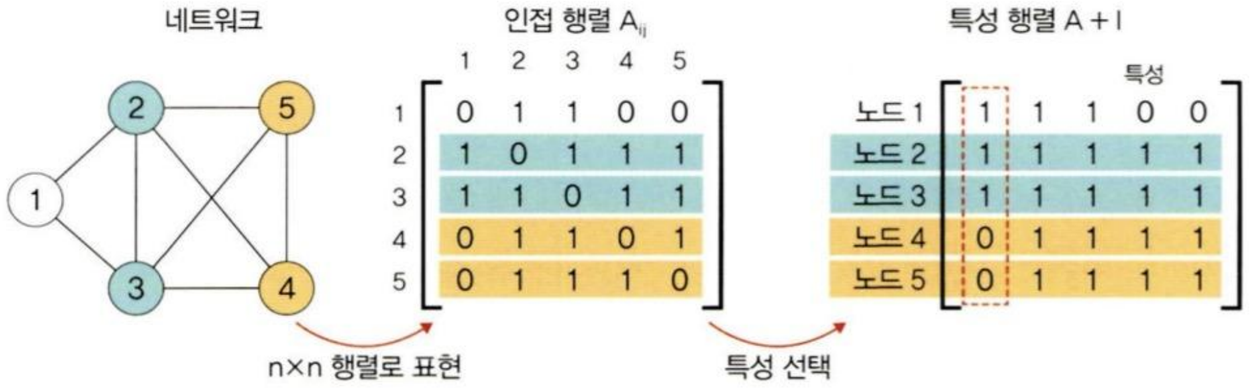
1단계 : 인접 행렬(adjacency matrix)

- 노드 n개를 n x n 행렬로 표현
- 인접 행렬 내의 값 : A_{ij}에 노드 i와 j의 관련성 여부

-> 그래프의 연결 관계 표현

2단계 : 특성 행렬(feature matrix)

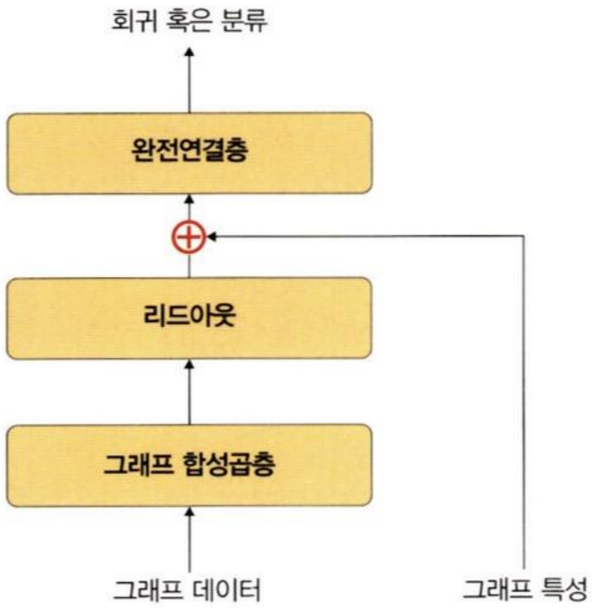
- 각 입력 데이터에서 이용할 특성 선택
- 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미
- 교재 예시: 인접 행렬 A에 단위 행렬 I를 더한 A+I 사용



#04 그래프 합성곱 네트워크

GCN(Graph Convolutional Network)

: 이미지에 대한 **합성곱을 그래프 데이터로 확장한** 알고리즘



그래프 합성곱층

- 그래프 형태의 데이터를 **행렬 형태로 변환**

-> 일반적인 딥러닝 알고리즘을 적용할 수 있도록 함

리드아웃(readout)

- 노드의 특성 행렬을 **하나의 벡터**로 변환
- 예시: 전체 노드 특성 벡터의 평균
- **그래프 전체**를 표현하는 벡터 생성

EWHA EURON

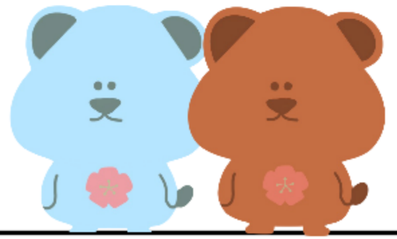
#05 그래프 합성곱 네트워크

GCN의 활용 분야

- SNS에서의 관계 네트워크
- 학술 연구에서의 인용 네트워크
- 3D Mesh : 3D 물체의 표면을 노드와 에지의 연결 구조로 표현한 데이터

EWHA EURON

THANK YOU



EWHA EURON